

Experiment 01 · UART – Asynchronous Serial Link

Two NUCLEO-F446RE boards exchanging button commands over USART1 in asynchronous mode

NUCLEO-F446RE
× 2
STM32F446RET6,
Cortex-M4

USART1 (APB2)
peripheral used

9600 Bd, 8N1
line settings

3 (TX, RX, GND)
wires between boards

Bare metal
CMSIS registers, no
HAL, no CubeMX

1 Aim and specification

Two NUCLEO-F446RE boards are made to talk to each other over **UART** only. The board programmed as **master** reads its blue B1 user button and measures how long it is held; the board programmed as **slave** has no button and only reacts to what arrives on the UART link, blinking its green LD2 LED in one of two clearly distinguishable patterns. After programming, both boards run from any 5 V USB supply with no PC attached.

1.1 Behavioural specification

Stimulus on the master	Measured press	Command transmitted	Slave LD2 pattern	LED-to-LED spacing	Total
Short click	< 1000 ms	0xA1 CMD_SHORT	3 blinks · 150 ms ON / 150 ms OFF	300 ms	0.9 s
Press and hold	≥ 1000 ms	0xA2 CMD_LONG	3 blinks · 1000 ms ON / 1000 ms OFF	2000 ms	6.0 s

The long-press pattern is the point of the experiment: 1000 ms on plus 1000 ms off means the slave LED starts a blink every **2.0 s**, against 300 ms for a short click. Both numbers are single `#define`s at the top of the slave listing (`LONG_ON_MS` , `LONG_OFF_MS` , `LONG_REPEAT`), so the timing can be retuned without touching the protocol code.

1.2 Status indication on the master

Indication	When	Meaning
Blue	steady while the frame is on the wire	transmission in progress
Green + one 250 ms LD2 flash	after a valid acknowledgement	the slave received the command
Red + three fast LD2 flashes	acknowledgement missing or wrong	wiring, power or protocol fault

The RGB LED is optional - set `USE_RGB` to 0 in the master listing and the on-board green LD2 alone carries the same information. If your RGB LED is common-anode, set `RGB_COMMON_CATHODE` to 0 instead of rewiring.

1.3 Frame format

All four experiments in this series use the same three-byte payload so that the protocols can be compared fairly:

Byte	Value	Purpose
1	0xAA	sync pattern - alternating bits, easy to recognise on an oscilloscope and unlikely to appear in noise
2	0xA1 or 0xA2	the command itself
3	~cmd (0x5E / 0x5D)	one's-complement checksum; the receiver acts only if it matches

2 Apparatus

Qty	Item	Notes
2	NUCLEO-F446RE development board	one programmed as master, one as slave
1 set	Male-male jumper wires (Dupont)	keep them under ~20 cm
2	USB Mini-B cables	for programming, then for standalone power
1-2	USB power bank or 5 V phone charger	to run the boards away from the PC
1	Common-cathode RGB LED + 3 × 330 Ω	optional master status indicator
1	Small breadboard	only needed for the RGB LED / pull-up resistors

3 Pin assignment

MCU pin	Function	Configuration	Morpho	Arduino	Used by
PA9	USART1_TX	AF7, push-pull	CN10-21	D8	master → slave data
PA10	USART1_RX	AF7, pull-up	CN10-33	D2	slave → master ACK
PC13	B1 user button	input, pull-up	CN7-23	–	master only
PA5	LD2 user LED	output, push-pull	CN10-11	D13	both boards
PC0/PC1/PC2	RGB R/G/B	output, push-pull	CN7-38/36/35	–	optional, master only
GND	0 V reference	–	CN10-20	–	mandatory

Reading the headers

CN7 and CN10 are the two 2×19 “morpho” headers along the long edges. Pin 1 of each is the row nearest the ST-LINK part of the board, odd numbers are in the left column and even numbers in the right column when the board is seen from above with the USB connector upwards. Every pin name is printed on the underside of the PCB – when in doubt, identify a pin by its *name*, not by counting.

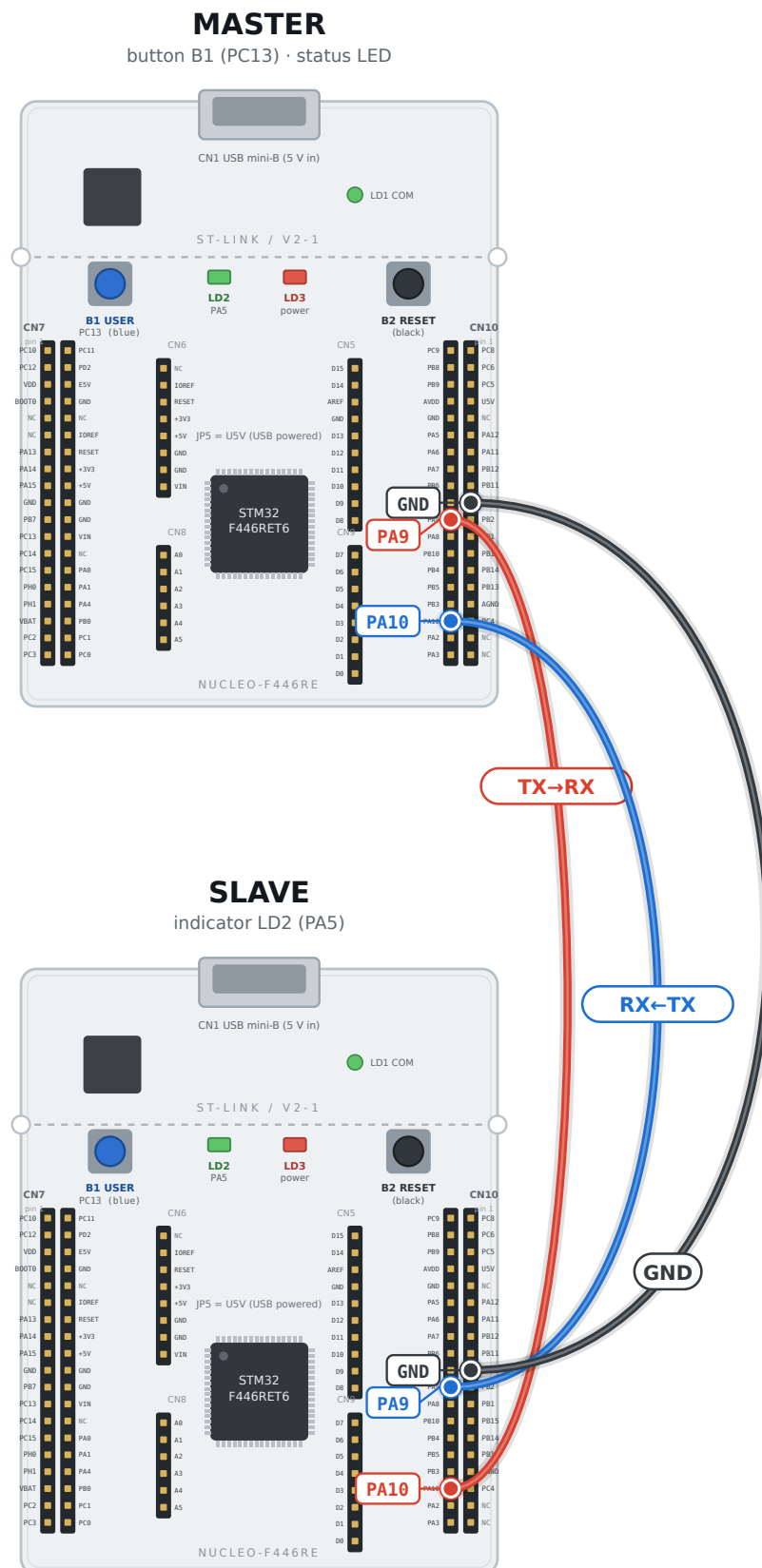
4 Circuit and wiring

Make every connection with the boards **unpowered**. The colours in the last column are the ones used in Figure 1 on the next page; any colour works electrically, but keeping to a convention makes a wiring fault obvious at a glance.

Signal	MASTER pin	SLAVE pin	Slave function	Wire colour used	Topology
USART1_TX	PA9 · CN10-21	PA10 · CN10-33	USART1_RX	red	crossed
USART1_RX	PA10 · CN10-33	PA9 · CN10-21	USART1_TX	blue	crossed
GND	GND · CN10-20	GND · CN10-20	GND	black	common

Experiment 1 · UART (USART1 asynchronous) - board-to-board wiring

Top view, USB connectors upwards. Pin names are silk-screened on the underside of each board.



TX and RX are CROSSED. The two grounds must be joined, or the receivers have no reference for the logic level.
PA9 = CN10-21 = Arduino D8 | PA10 = CN10-33 = Arduino D2 | GND = CN10-20.

Figure 1 - Physical wiring between the two boards, drawn to the real board outline. The coloured rings mark the pads that carry a wire.

5 Theory of operation

An asynchronous UART link carries **no clock**. The two ends only agree on a bit period in advance; the receiver re-synchronises on the falling edge of every start bit and then samples each bit at its centre using a clock 16 times faster than the bit rate. That is why the baud rate must be programmed identically on both boards – an error of more than about $\pm 2\%$ corrupts the last bits of every character.

The bit period is set by the `BRR` register, which holds the value $f_{\text{PCLK2}} / \text{baud}$ in a 12.4 fixed-point format:

$$\text{BRR} = 16\,000\,000 / 9600 = 1666.67 \approx 1667 = 0x0683$$

The resulting baud rate is $16\,000\,000 / 1667 = 9598$ Bd, an error of 0.02 % – far inside the tolerance. Because the whole design runs from the internal 16 MHz RC oscillator (HSI) with all bus prescalers set to /1, PCLK2 is exactly 16 MHz and no PLL configuration is needed.

Why a three-byte frame?

A single byte on a bare wire cannot be validated. The firmware therefore sends `0xAA` (a sync pattern with alternating bits, easy to spot on a scope), the command, and the command's one's complement as a checksum. The receiver runs a three-state machine and only acts when all three bytes agree, so electrical noise on the line cannot make the LED blink by accident.

Flow of one press

1. The master de-bounces B1, measures the press with the 1 ms SysTick counter and classifies it as short or long.
2. It writes the three bytes into `USART1->DR`, waiting for `TXE` before each one and for `TC` after the last.
3. The slave's state machine validates the frame, immediately transmits `0x5A` so the master is not held up, and only then runs the blink pattern.
4. The master waits up to 500 ms for that acknowledgement and shows green or red.

5.1 Clock configuration used by both boards

Both listings begin with `clock_init_hsi_16mhz()`, which selects the internal 16 MHz RC oscillator, leaves every bus prescaler at /1 and sets zero flash wait states. Nothing is left to a start-up file or to CubeMX, so **SYSCCLK = HCLK = PCLK1 = PCLK2 = 16 MHz** and every timing number in this manual can be verified by hand. `SysTick` is then reloaded with 15999 to interrupt once per millisecond, which is the time base for both the button measurement and the blink pattern.

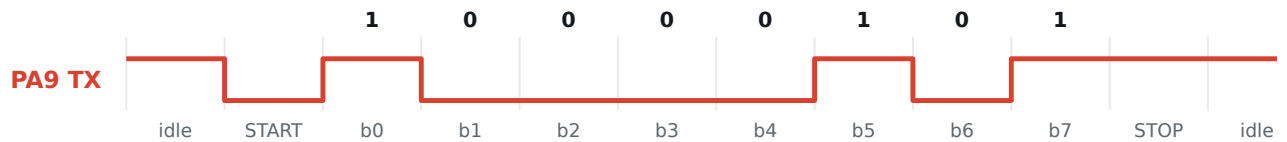
5.2 Register configuration summary

Register	Value written	Why
<code>RCC->AHB1ENR</code>	<code> = GPIOAEN</code>	clock to GPIOA – a peripheral with no clock ignores every write
<code>RCC->APB2ENR</code>	<code> = USART1EN</code>	clock to USART1
<code>GPIOA->MODER</code>	<code>PA9, PA10 = 10</code>	alternate-function mode instead of plain GPIO
<code>GPIOA->AFR[1]</code>	<code>PA9, PA10 = 7</code>	AF7 routes the pins to USART1
<code>GPIOA->PUPDR</code>	<code>PA9, PA10 = 01</code>	pull-ups keep the idle level high when a wire is loose
<code>USART1->BRR</code>	<code>1667</code>	9600 Bd from a 16 MHz PCLK2
<code>USART1->CR1</code>	<code>UE TE RE</code>	enable the peripheral, the transmitter and the receiver
<code>USART1->CR2</code>	<code>0</code>	1 stop bit, no clock output (this is what makes it asynchronous)
<code>USART1->SR</code>	<code>poll TXE, TC, RXNE, ORE</code>	transmit-empty, transmission-complete, receive-ready, overrun

USART1->DR	read / write	the single 9-bit data register; reading it clears RXNE
------------	--------------	--

6 Signal timing

USART1 asynchronous character - 8N1, 9600 Bd, data byte 0xA1



0xA1 = 1010 0001 is shifted out LSB first, so the line carries 1 0 0 0 0 1 0 1.
 One bit lasts $1/9600 \text{ s} = 104.2 \mu\text{s}$; the complete 10-bit character takes about 1.04 ms.

Figure 2 – One asynchronous character on PA9. There is no clock wire; the receiver recovers the bit boundaries from the falling edge of the start bit and its own 16×-oversampled baud-rate generator.

7 MASTER source code `uart_master.c`

Paste this whole listing over the contents of `Core/Src/main.c` in the master project. It is self-contained: there are no other files to add and no header to edit.

`uart_master.c`

```
1/*=====
2 * EXPERIMENT 1 : UART (USART1, ASYNCHRONOUS)          --- M A S T E R ---
3 * Board : NUCLEO-F446RE (STM32F446RET6, Cortex-M4)
4 * Style : 100 % bare-metal CMSIS register programming - no HAL, no CubeMX.
5 *       Everything (clock tree, GPIO, USART, SysTick) is configured here.
6 *
7 * FUNCTION
8 *   B1 (blue user button, PC13) is sampled and de-bounced in software.
9 *   The press LENGTH is measured with a 1 ms SysTick time base:
10 *     press < 1000 ms -> transmit CMD_SHORT (0xA1)
11 *     press >= 1000 ms -> transmit CMD_LONG (0xA2)
12 *   Frame sent : [0xAA sync][cmd][~cmd checksum]
13 *   The slave answers with ACK (0x5A); the master then flashes LD2 green
14 *   (and the optional external RGB LED green = OK / red = no answer).
15 *
16 * WIRING (master -> slave)
17 *   PA9 USART1_TX (CN10-21 / D8) -> PA10 USART1_RX (CN10-33 / D2)
18 *   PA10 USART1_RX (CN10-33 / D2) <- PA9 USART1_TX (CN10-21 / D8)
19 *   GND (CN10-20) <-> GND (CN10-20)
20 *
21 * Optional RGB status LED on the MASTER (common cathode + 3 x 330 R):
22 *   PC0 = RED (CN7-38), PC1 = GREEN (CN7-36), PC2 = BLUE (CN7-35),
23 *   common cathode -> GND (CN7-20). Set USE_RGB to 0 if not fitted.
24 *=====*/
25
26#include "stm32f446xx.h"
27#include <stdint.h>
28
29/*----- user configuration -----*/
30#define USE_RGB 1u /* 1 = external RGB LED fitted on PC0/1/2 */
31#define RGB_COMMON_CATHODE 1u /* 0 if you use a common-ANODE RGB LED */
32
33#define LED_PIN 5u /* PA5 - LD2 green user LED */
34#define BTN_PIN 13u /* PC13 - B1 blue user button (active 0) */
35
36#define CMD_SYNC 0xAAu
37#define CMD_SHORT 0xA1u
38#define CMD_LONG 0xA2u
39#define ACK_BYTE 0x5Au
40
41#define T_DEBOUNCE_MS 20u /* mechanical bounce window */
42#define T_LONG_MS 1000u /* short / long press decision threshold */
43#define T_ACK_MS 500u /* how long we wait for the slave's ACK */
44
45/*===== 1. CLOCK : HSI 16 MHz, no PLL =====*/
46/* Deterministic and simple: SYSCLK = HCLK = PCLK1 = PCLK2 = 16 MHz. */
47static void clock_init_hsi_16mhz(void)
48{
49    RCC->CR |= RCC_CR_HSION; /* start internal 16 MHz RC */
50    while ((RCC->CR & RCC_CR_HSIRDY) == 0u) { }
51
52    FLASH->ACR = FLASH_ACR_LATENCY_0WS; /* 16 MHz @3.3 V -> 0 WS */
53
54    RCC->CFGR &= ~RCC_CFGR_SW; /* SW = 00 -> HSI is SYSCLK */
55    while ((RCC->CFGR & RCC_CFGR_SWS) != RCC_CFGR_SWS_HSI) { }
56
57    RCC->CFGR &= ~(RCC_CFGR_HPRE | RCC_CFGR_PPRE1 | RCC_CFGR_PPRE2);
58    /* AHB /1, APB1 /1, APB2 /1 -> PCLK1 = PCLK2 = 16 MHz */
59}
60
61/*===== 2. SysTick 1 ms time base =====*/
62static volatile uint32_t s_ticks;
63
```

```

64void SysTick_Handler(void) { s_ticks++; }          /* weak symbol in startup_.s */
65
66static void systick_init(void)
67{
68    SysTick->LOAD = 16000u - 1u;                    /* 16 MHz / 16000 = 1 kHz */
69    SysTick->VAL = 0u;
70    SysTick->CTRL = SysTick_CTRL_CLKSOURCE_Msk |
71                  SysTick_CTRL_TICKINT_Msk |
72                  SysTick_CTRL_ENABLE_Msk;
73}
74static uint32_t millis(void) { return s_ticks; }
75static void delay_ms(uint32_t ms)
76{
77    uint32_t t0 = s_ticks;
78    while ((uint32_t)(s_ticks - t0) < ms) { __NOP(); }
79}
80
81/*===== 3. On-board LD2 =====*/
82static void led_init(void)
83{
84    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;
85    (void)RCC->AHB1ENR;                             /* clock-enable read-back */
86
87    GPIOA->MODER  &= ~(3u << (LED_PIN * 2u));
88    GPIOA->MODER  |= (1u << (LED_PIN * 2u));        /* 01 = general output */
89    GPIOA->OTYPER &= ~(1u << LED_PIN);              /* push-pull */
90    GPIOA->OSPEEDR &= ~(3u << (LED_PIN * 2u));     /* low speed is fine */
91    GPIOA->PUPDR  &= ~(3u << (LED_PIN * 2u));
92    GPIOA->BSRR   = (1u << (LED_PIN + 16u));        /* start OFF */
93}
94static void led_on (void) { GPIOA->BSRR = (1u << LED_PIN); }
95static void led_off(void) { GPIOA->BSRR = (1u << (LED_PIN + 16u)); }
96
97/*===== 4. On-board B1 =====*/
98static void button_init(void)
99{
100    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOCEN;
101    (void)RCC->AHB1ENR;
102
103    GPIOC->MODER &= ~(3u << (BTN_PIN * 2u));        /* 00 = input */
104    GPIOC->PUPDR &= ~(3u << (BTN_PIN * 2u));
105    GPIOC->PUPDR |= (1u << (BTN_PIN * 2u));          /* 01 = internal pull-up */
106}
107static uint32_t button_down(void)
108{
109    return ((GPIOC->IDR & (1u << BTN_PIN)) == 0u); /* B1 is active LOW */
110}
111
112/* Blocking, de-bounced press capture.
113 * Returns 0 (nothing), CMD_SHORT or CMD_LONG and reports the press length. */
114static uint8_t button_capture(uint32_t *press_ms)
115{
116    uint32_t t0, d;
117
118    if (!button_down()) return 0u;
119    delay_ms(T_DEBOUNCE_MS);
120    if (!button_down()) return 0u; /* it was only a bounce */
121
122    t0 = millis();
123    while (button_down()) { __NOP(); } /* wait for the release */
124    delay_ms(T_DEBOUNCE_MS); /* de-bounce the release */
125
126    d = millis() - t0;
127    if (press_ms != 0) { *press_ms = d; }
128    return (d >= T_LONG_MS) ? CMD_LONG : CMD_SHORT;
129}
130
131/*===== 5. Optional RGB indicator =====*/
132static void rgb_init(void)
133{
134#ifdef USE_RGB
135    uint32_t p;

```

```

136 RCC->AHB1ENR |= RCC_AHB1ENR_GPIOCEN;
137 (void)RCC->AHB1ENR;
138 for (p = 0u; p <= 2u; p++) { /* PC0 = R, PC1 = G, PC2 = B */
139     GPIOC->MODER &= ~(3u << (p * 2u));
140     GPIOC->MODER |= (1u << (p * 2u));
141     GPIOC->OTYPER &= ~(1u << p);
142     GPIOC->PUPDR &= ~(3u << (p * 2u));
143 }
144#endif
145}
146static void rgb_set(uint32_t r, uint32_t g, uint32_t b)
147{
148    #if USE_RGB
149    #if RGB_COMMON_CATHODE
150        GPIOC->BSRR = (r ? (1u << 0) : (1u << 16)) |
151                      (g ? (1u << 1) : (1u << 17)) |
152                      (b ? (1u << 2) : (1u << 18));
153    #else /* common anode: inverted */
154        GPIOC->BSRR = (r ? (1u << 16) : (1u << 0)) |
155                      (g ? (1u << 17) : (1u << 1)) |
156                      (b ? (1u << 18) : (1u << 2));
157    #endif
158    #else
159        (void)r; (void)g; (void)b;
160    #endif
161}
162
163/* Green + one long LD2 flash = ACK received. Red + triple flash = no ACK. */
164static void master_indicate(uint32_t ok)
165{
166    uint32_t i;
167    if (ok) {
168        rgb_set(0u, 1u, 0u);
169        led_on(); delay_ms(250u); led_off();
170    } else {
171        rgb_set(1u, 0u, 0u);
172        for (i = 0u; i < 3u; i++) { led_on(); delay_ms(60u); led_off(); delay_ms(60u); }
173    }
174    rgb_set(0u, 0u, 0u);
175}
176
177/*===== 6. USART1 =====*/
178/* PA9 = USART1_TX (AF7) PA10 = USART1_RX (AF7)
179 * 9600 baud, 8 data bits, no parity, 1 stop bit, oversampling by 16.
180 * BRR = fPCLK2 / baud = 16 000 000 / 9600 = 1666.67 -> 1667 (0x683) */
181static void usart1_init(void)
182{
183    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;
184    RCC->APB2ENR |= RCC_APB2ENR_USART1EN;
185    (void)RCC->APB2ENR;
186
187    GPIOA->MODER &= ~(3u << (9u * 2u)) | (3u << (10u * 2u));
188    GPIOA->MODER |= ((2u << (9u * 2u)) | (2u << (10u * 2u))); /* 10 = AF */
189    GPIOA->OTYPER &= ~(1u << 9u) | (1u << 10u); /* push-pull */
190    GPIOA->OSPEEDR |= ((3u << (9u * 2u)) | (3u << (10u * 2u))); /* high sp. */
191    GPIOA->PUPDR &= ~(3u << (9u * 2u)) | (3u << (10u * 2u));
192    GPIOA->PUPDR |= ((1u << (9u * 2u)) | (1u << (10u * 2u))); /* pull-up */
193
194    /* AFR[1] = AFRH, 4 bits per pin, pin n uses bits (n-8)*4 */
195    GPIOA->AFR[1] &= ~(0xFu << ((9u - 8u) * 4u)) | (0xFu << ((10u - 8u) * 4u));
196    GPIOA->AFR[1] |= ((7u << ((9u - 8u) * 4u)) | (7u << ((10u - 8u) * 4u)));
197
198    USART1->CR1 = 0u; /* disable while (re)configuring */
199    USART1->CR2 = 0u; /* 1 stop bit, no clock output */
200    USART1->CR3 = 0u; /* no flow control, no DMA */
201    USART1->BRR = 1667u; /* 9600 Bd @ 16 MHz */
202    USART1->CR1 = USART_CR1_TE | USART_CR1_RE | USART_CR1_UE;
203}
204
205static void usart1_write(uint8_t b)
206{
207    while ((USART1->SR & USART_SR_TXE) == 0u) { /* TDR empty ? */

```

```

208     USART1->DR = (uint32_t)b;
209 }
210 static void usart1_flush(void)
211 {
212     while ((USART1->SR & USART_SR_TC) == 0u) { }    /* last stop bit gone    */
213 }
214 /* Returns 1 and the byte, or 0 on timeout. */
215 static uint32_t usart1_read(uint8_t *b, uint32_t timeout_ms)
216 {
217     uint32_t t0 = millis();
218     while ((uint32_t)(millis() - t0) < timeout_ms) {
219         if (USART1->SR & USART_SR_RXNE) { *b = (uint8_t)USART1->DR; return 1u; }
220         if (USART1->SR & USART_SR_ORE) { (void)USART1->SR; (void)USART1->DR; }
221     }
222     return 0u;
223 }
224
225 /*===== main =====*/
226 int main(void)
227 {
228     uint8_t cmd, ack;
229     uint32_t press_ms, ok;
230
231     clock_init_hsi_16mhz();
232     systick_init();
233     led_init();
234     button_init();
235     rgb_init();
236     usart1_init();
237
238     rgb_set(0u, 0u, 1u); led_on(); delay_ms(300u); led_off(); rgb_set(0u,0u,0u);
239
240     for (;;) {
241         press_ms = 0u;
242         cmd = button_capture(&press_ms);
243         if (cmd == 0u) { continue; }
244
245         rgb_set(0u, 0u, 1u);                /* blue = transmitting    */
246
247         usart1_write(CMD_SYNC);
248         usart1_write(cmd);
249         usart1_write((uint8_t)(cmd ^ 0xFFu)); /* checksum                */
250         usart1_flush();
251
252         rgb_set(0u, 0u, 0u);
253
254         ack = 0u;
255         ok = usart1_read(&ack, T_ACK_MS) && (ack == ACK_BYTE);
256         master_indicate(ok);
257     }
258 }

```

8 SLAVE source code `uart_slave.c`

Paste this whole listing over the contents of `Core/Src/main.c` in the slave project. It is self-contained: there are no other files to add and no header to edit.

`uart_slave.c`

```
1/*=====
2 *  EXPERIMENT 1 : UART (USART1, ASYNCHRONOUS)          --- S L A V E ---
3 *  Board   : NUCLEO-F446RE (STM32F446RET6, Cortex-M4)
4 *  Style   : 100 % bare-metal CMSIS register programming - no HAL, no CubeMX.
5 *
6 *  FUNCTION
7 *      Continuously parses the 3-byte frame [0xAA][cmd][~cmd] on USART1.
8 *      When a valid frame arrives it immediately answers ACK (0x5A) so the
9 *      master is not blocked, and then plays the LD2 (PA5) blink pattern:
10 *
11 *      CMD_SHORT (0xA1)  -> 3 blinks, 150 ms ON / 150 ms OFF
12 *                          (LED turns on every 300 ms)
13 *      CMD_LONG  (0xA2)  -> 3 blinks, 1000 ms ON / 1000 ms OFF
14 *                          (LED turns on every 2000 ms = 2 s spacing)
15 *
16 *  WIRING (see master file - the two boards are wired TX<->RX crossed, and
17 *  the two grounds MUST be joined).
18 *=====*/
19
20#include "stm32f446xx.h"
21#include <stdint.h>
22
23#define LED_PIN      5u          /* PA5 - LD2 green user LED          */
24
25#define CMD_SYNC      0xAAu
26#define CMD_SHORT     0xA1u
27#define CMD_LONG      0xA2u
28#define ACK_BYTE      0x5Au
29
30/* ---- formal blink specification (edit here to retune the experiment) ---- */
31#define SHORT_ON_MS    150u
32#define SHORT_OFF_MS   150u
33#define SHORT_REPEAT    3u
34#define LONG_ON_MS     1000u
35#define LONG_OFF_MS     1000u    /* 1000 + 1000 = 2000 ms LED-to-LED period */
36#define LONG_REPEAT     3u
37
38/*===== 1. CLOCK : HSI 16 MHz, no PLL =====*/
39static void clock_init_hsi_16mhz(void)
40{
41    RCC->CR |= RCC_CR_HSION;
42    while ((RCC->CR & RCC_CR_HSIRDY) == 0u) { }
43
44    FLASH->ACR = FLASH_ACR_LATENCY_0WS;
45
46    RCC->CFGR &= ~RCC_CFGR_SW;
47    while ((RCC->CFGR & RCC_CFGR_SWS) != RCC_CFGR_SWS_HSI) { }
48
49    RCC->CFGR &= ~(RCC_CFGR_HPRE | RCC_CFGR_PPRE1 | RCC_CFGR_PPRE2);
50}
51
52/*===== 2. SysTick 1 ms time base =====*/
53static volatile uint32_t s_ticks;
54
55void SysTick_Handler(void) { s_ticks++; }
56
57static void systick_init(void)
58{
59    SysTick->LOAD = 16000u - 1u;
60    SysTick->VAL  = 0u;
61    SysTick->CTRL = SysTick_CTRL_CLKSOURCE_Msk |
62                    SysTick_CTRL_TICKINT_Msk  |
63                    SysTick_CTRL_ENABLE_Msk;
```

```

64}
65static uint32_t millis(void) { return s_ticks; }
66static void delay_ms(uint32_t ms)
67{
68    uint32_t t0 = s_ticks;
69    while ((uint32_t)(s_ticks - t0) < ms) { __NOP(); }
70}
71
72/*===== 3. On-board LD2 =====*/
73static void led_init(void)
74{
75    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;
76    (void)RCC->AHB1ENR;
77
78    GPIOA->MODER  &= ~(3u << (LED_PIN * 2u));
79    GPIOA->MODER  |= (1u << (LED_PIN * 2u));
80    GPIOA->OTYPER  &= ~(1u << LED_PIN);
81    GPIOA->OSPEEDR &= ~(3u << (LED_PIN * 2u));
82    GPIOA->PUPDR  &= ~(3u << (LED_PIN * 2u));
83    GPIOA->BSRR   = (1u << (LED_PIN + 16u));
84}
85static void led_on (void) { GPIOA->BSRR = (1u << LED_PIN); }
86static void led_off(void) { GPIOA->BSRR = (1u << (LED_PIN + 16u)); }
87
88/* The behaviour that the whole experiment is about. */
89static void run_pattern(uint8_t cmd)
90{
91    uint32_t i, n, on, off;
92
93    if (cmd == CMD_LONG) { n = LONG_REPEAT; on = LONG_ON_MS; off = LONG_OFF_MS; }
94    else                  { n = SHORT_REPEAT; on = SHORT_ON_MS; off = SHORT_OFF_MS; }
95
96    for (i = 0u; i < n; i++) {
97        led_on(); delay_ms(on);
98        led_off(); delay_ms(off);
99    }
100}
101
102/*===== 4. USART1 =====*/
103static void usart1_init(void)
104{
105    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;
106    RCC->APB2ENR |= RCC_APB2ENR_USART1EN;
107    (void)RCC->APB2ENR;
108
109    GPIOA->MODER  &= ~((3u << (9u * 2u)) | (3u << (10u * 2u)));
110    GPIOA->MODER  |= ((2u << (9u * 2u)) | (2u << (10u * 2u)));
111    GPIOA->OTYPER  &= ~(1u << 9u) | (1u << 10u);
112    GPIOA->OSPEEDR |= ((3u << (9u * 2u)) | (3u << (10u * 2u)));
113    GPIOA->PUPDR  &= ~((3u << (9u * 2u)) | (3u << (10u * 2u)));
114    GPIOA->PUPDR  |= ((1u << (9u * 2u)) | (1u << (10u * 2u)));
115
116    GPIOA->AFR[1] &= ~((0xFu << ((9u - 8u) * 4u)) | (0xFu << ((10u - 8u) * 4u)));
117    GPIOA->AFR[1] |= ((7u << ((9u - 8u) * 4u)) | (7u << ((10u - 8u) * 4u)));
118
119    USART1->CR1 = 0u;
120    USART1->CR2 = 0u;
121    USART1->CR3 = 0u;
122    USART1->BRR = 1667u; /* 9600 Bd @ 16 MHz - must match! */
123    USART1->CR1 = USART_CR1_TE | USART_CR1_RE | USART_CR1_UE;
124}
125
126static void usart1_write(uint8_t b)
127{
128    while ((USART1->SR & USART_SR_TXE) == 0u) { }
129    USART1->DR = (uint32_t)b;
130}
131static void usart1_flush(void)
132{
133    while ((USART1->SR & USART_SR_TC) == 0u) { }
134}
135static uint32_t usart1_read(uint8_t *b, uint32_t timeout_ms)

```

```

136{
137    uint32_t t0 = millis();
138    while ((uint32_t)(millis() - t0) < timeout_ms) {
139        if (USART1->SR & USART_SR_RXNE) { *b = (uint8_t)USART1->DR; return 1u; }
140        if (USART1->SR & USART_SR_ORE) { (void)USART1->SR; (void)USART1->DR; }
141    }
142    return 0u;
143}
144
145/*===== main =====*/
146int main(void)
147{
148    uint8_t byte, cmd = 0u;
149    uint32_t state = 0u;          /* 0 = wait sync, 1 = wait cmd, 2 = chk */
150
151    clock_init_hsi_16mhz();
152    systick_init();
153    led_init();
154    usart1_init();
155
156    led_on(); delay_ms(300u); led_off();          /* "slave alive" signal */
157
158    for (;;) {
159        if (!usart1_read(&byte, 10u)) { continue; }
160
161        switch (state) {
162            case 0u:                      /* hunting for the sync */
163                if (byte == CMD_SYNC) { state = 1u; }
164                break;
165
166            case 1u:                      /* command byte */
167                cmd = byte;
168                state = 2u;
169                break;
170
171            default:                      /* checksum byte */
172                if ((uint8_t)(cmd ^ 0xFFu) == byte &&
173                    (cmd == CMD_SHORT || cmd == CMD_LONG)) {
174                    usart1_write(ACK_BYTE);          /* answer first ... */
175                    usart1_flush();
176                    run_pattern(cmd);                /* ... then blink */
177                }
178                state = 0u;
179                break;
180        }
181    }
182}

```

9 Building it in STM32CubeIDE without touching CubeMX

1. **File → New → STM32 Project.** In the target selector open the *Board Selector* tab, type `NUCLEO-F446RE` and select it (or use the *MCU/MPU Selector* and pick `STM32F446RET6`).
2. Give the project a name, keep *Targeted Language* = **C** and *Targeted Binary Type* = **Executable**, and – this is the important one – set **Targeted Project Type = Empty**. An *Empty* project gives you the startup assembly file, the linker script, `system_stm32f4xx.c` and the CMSIS register headers, but **no HAL, no LL drivers and no .ioc file**. There is therefore nothing to configure graphically – which is exactly what a bare-metal experiment requires.
3. Answer **No** if the IDE offers to initialise peripherals; with an *Empty* project that dialog normally does not appear at all.
4. Open `Core/Src/main.c`, delete its whole contents and paste the listing from this manual. The single include `#include "stm32f446xx.h"` resolves through `Drivers/CMSIS/Device/ST/STM32F4xx/Include`, which the Empty project already put on the include path.
5. The symbol `STM32F446xx` is predefined by the project wizard. If the compiler complains that the header cannot decide which device it is, add it manually under *Project → Properties → C/C++ Build → Settings → MCU GCC Compiler → Preprocessor*.
6. Build with **Ctrl + B**. Connect the first board, then *Run → Run As → STM32 C/C++ Application* to flash it.
7. Repeat steps 1–6 in a second project for the other listing, and flash it to the second board. Label the boards **MASTER** and **SLAVE** with a marker – they look identical once they are off the PC.

Alternative without the IDE: copy the generated `.bin` onto the `NODE_F446RE` mass-storage drive that appears when the board is plugged in, or use STM32CubeProgrammer.

10 Running the pair with no PC attached

Nothing in these programs depends on the debugger. The moment the board is powered, the reset vector in flash runs `main()`, so the two boards work as a stand-alone pair.

Option A – one USB supply per board (recommended)

1. Unplug both boards from the PC after flashing.
2. Plug each board's CN1 Mini-B socket into a USB power bank, a 5 V phone charger or a powered USB hub. Leave jumper **JP5 on U5V** (its factory position).
3. LD3 (red) lights up on each board and both LD2s give a single ~300 ms flash: that is the power-up self-test built into the firmware.
4. Keep the protocol wires and the common ground connected exactly as during programming.

Option B – one 5 V source for both boards

1. Move **JP5 to E5V** on *both* boards.
2. Feed +5 V into `E5V` (CN7 pin 6) and 0 V into `GND` (CN7 pin 8 or CN10 pin 20) of both boards from the same supply.
3. The grounds are then already common, so the separate GND jumper of the protocol wiring becomes redundant – but leaving it in place does no harm.

Safety

Never drive 5 V into the `+5V` pin while JP5 is on U5V and a USB cable is plugged in – you would connect two supplies together. Choose one powering scheme and stay with it. BOOT0 must remain low (its default) so the MCU boots from flash.

11 Test procedure and observations

1. Flash the master listing to board 1 and the slave listing to board 2.
2. Power both boards down, then wire them exactly as in the connection table. Check the ground wire twice – it is the single most common cause of a dead link.
3. Power both boards. Each LD2 must give one short flash: both CPUs are alive.
4. **Test 1 – short click.** Press and release B1 on the master quickly (< 1 s). The master should flash green/LD2 and the slave should give three fast blinks.
5. **Test 2 – long press.** Hold B1 for at least one second, then release. The slave must now blink three times with one second on and one second off, i.e. two seconds between the start of one blink and the start of the next.
6. Time the slow pattern with a stopwatch and record it. Repeat each test three times.
7. **Test 3 – error case.** Pull the data wire out and press the button again. The master must show the red / triple-flash failure indication instead of green.
8. If an oscilloscope or logic analyser is available, capture the line(s) during one press and compare the capture with the timing figure in section 6.

Trial	Press duration (ms)	Expected command	Master indication	Blinks counted	Measured spacing (s)	Pass
1						
2						
3						
4						
5						
6						

12 Troubleshooting

Symptom	Cause and remedy
Characters arrive but are garbage	The two boards are not at the same baud rate, or one of them still runs from a PLL clock. Check that <code>BRR = 1667</code> in both listings and that neither project calls a HAL clock initialiser.
Master shows red although the slave blinks	The return wire (slave PA9 → master PA10) is missing. Only the ACK path is broken, so the command still gets through.
Neither LD2 flashes at power-up	The board is not running: check LD3 (red) is lit, that the USB supply really delivers power, and that you flashed the board you think you flashed.
The slave never reacts, master always shows red	Ground is not common. Join GND (CN10-20) of the two boards.
Behaviour is erratic, blinks appear at random	Wires too long or badly seated. Use jumpers under 20 cm and push them fully home.
Both boards blink but the patterns are swapped	The master listing was flashed to the slave board. Label the boards.
Long press is detected as a short one	You released before 1000 ms. The threshold is <code>T_LONG_MS</code> in the master listing – change it there if your lab asks for a different value.

13 Review questions

1. Why does an asynchronous link need a start bit at all?

2. Compute BRR for 115200 Bd with PCLK2 = 16 MHz. What is the resulting error?
3. What is the difference between the TXE and TC flags?
4. What causes the ORE flag and how is it cleared on an STM32F4?
5. Why are TX and RX crossed between the two boards but SCK is not in SPI?
6. How many bits are actually on the wire for one 8N1 character?
7. What would happen if only the data wires were connected and not GND?
8. How would you make the receive path interrupt-driven instead of polled?